

UNIVERSIDAD AUTONOMA TOMAS FRIAS

Facultad de Ciencias Puras • Ingeniería Informática

INF761 — DESARROLLO DE VIDEOJUEGOS

Examen del Segundo Parcial • Gestión I-2026

Esquiva Geométrica

Construcción de un minijuego 2D aplicando Coroutines

Unity 6 • Solo figuras geométricas primitivas

Nombre del estudiante:**Registro / C.I.:****Fecha:**

Datos del examen	
Tiempo asignado	100 minutos
Modalidad	Practica individual en computadora
Puntaje total	100 puntos
Motor / version	Unity 6 (proyecto plantilla 2D)
Tema central	Coroutines (IEnumerator / yield)

Objetivo del examen

Demostrar el dominio de coroutines en Unity construyendo la logica temporal de un minijuego 2D que utiliza únicamente figuras geométricas (cuadrados, círculos y triángulos).

El docente entrega una base jugable ya funcional (Parte A). El estudiante debe completar las cuatro coroutines que faltan para que el juego funcione por completo (Parte B).

I. Indicaciones generales

1. Lee todo el examen antes de empezar. La Parte A es una guía que debes seguir paso a paso para dejar el juego base funcionando; no se califica, pero es indispensable para poder resolver la Parte B.
2. La Parte B contiene las cuatro coroutines que debes completar. Solo se evalúa el código de esos cuatro métodos y su correcto funcionamiento dentro del juego.
3. Puedes usar la documentación oficial de Unity. No se permite compartir código entre compañeros.
4. El juego usa exclusivamente figuras geométricas (sprites primitivos de Unity). No necesitas ningún recurso grafico externo.
5. Entrega: comprime la carpeta Assets/ del proyecto (sin la carpeta Library) en un archivo ZIP con el formato Apellido_Nombre_INF761_2P.zip.

Recordatorio de API (Unity 6)

Usa `rb.linearVelocity` en lugar de `rb.velocity` y `FindFirstObjectByType<T>()` en lugar de `FindObjectOfType<T>()`. En este examen el movimiento es por transform, así que basta con recordar la firma de las coroutines: deben devolver `IEnumerator` y arrancarse con `StartCoroutine(...)`.

II. Parte A — Guia: arma la base del juego (no se califica)

Sigue estos pasos para dejar el juego listo para jugar, salvo por las cuatro coroutines que completarás en la Parte B. Al terminar la Parte A el proyecto debe compilar sin errores.

Paso 1 — Crear el proyecto y la escena

1. Crea un proyecto nuevo con la plantilla 2D (Core) en Unity 6 y nómbralo EsquivaGeometrica.
2. Selecciona la Main Camera, ponla en Projection: Orthographic y Size: 6. Fondo a tu gusto (un color oscuro resalta las figuras).
3. Guarda la escena como Juego en una carpeta Assets/Scenes.

Paso 2 — Crear las figuras geométricas

Todas las figuras se crean desde GameObject > 2D Object > Sprites. No se usa ningún arte externo.

- Jugador: crea un Sprite Square. Renómbralo Jugador, escala (1,1,1), posicóñalo en la parte inferior (0, -4, 0) y dale un color claro (blanco o celeste).
- Obstáculo círculo: crea un Sprite Circle, color rojo. Sera un prefab.
- Obstáculo triángulo: crea un Sprite Triangle, color naranja. Sera un prefab.
- Poder: crea un Sprite Square, color verde, un poco más pequeño (escala 0.6). Sera un prefab (otorga escudo).

Paso 3 — Colliders, Rigidbody y Tags

1. Al Jugador agrégle: BoxCollider2D y un Rigidbody2D con Body Type = Kinematic. Asígnale el Tag "Player".
2. A cada obstáculo (círculo y triángulo) agrégle un Collider2D adecuado (CircleCollider2D o PolygonCollider2D) marcado como Is Trigger. Asígnale el Tag "Obstáculo".
3. Al Poder agregale un BoxCollider2D marcado como Is Trigger y el Tag "Poder".

Crear los Tags

Los tags "Obstáculo" y "Poder" no existen por defecto. Créalos en el Inspector con el desplegable Tag > Add Tag... antes de asignarlos. El Rigidbody2D Kinematic del Jugador es necesario para que se disparen los eventos OnTriggerEnter2D.

Paso 4 — Script Figura.cs (PROVISTO, ya incluye un ejemplo de coroutine)

Crea el script Figura.cs y pega exactamente este código. Luego asígnalo a los tres prefabs (círculo, triángulo y poder). Conviértelos en prefabs arrastrándolos a Assets/Prefabs y bórralos de la escena.

```
using System.Collections;
using UnityEngine;

public class Figura : MonoBehaviour
{
    [SerializeField] private float velocidadCaida = 3f;
    [SerializeField] private float duracionAparicion = 0.3f;

    private float limiteInferior = -6f;

    private void Start()
```

```

{
    // EJEMPLO RESUELTO de coroutine (ver mas abajo)
    StartCoroutine(RutinaAparicion());
}

private void Update()
{
    // Caída constante hacia abajo
    transform.Translate(Vector3.down * velocidadCaída * Time.deltaTime);

    // Autodestrucción al salir de la pantalla
    if (transform.position.y < limiteInferior)
        Destroy(gameObject);
}

// === EJEMPLO RESUELTO (referencia para el estudiante) ===
// La figura crece desde escala 0 hasta su tamaño normal
// en 'duracionAparicion' segundos, un poco cada frame.
private IEnumerator RutinaAparicion()
{
    Vector3 escalaFinal = transform.localScale;
    transform.localScale = Vector3.zero;

    float t = 0f;
    while (t < duracionAparicion)
    {
        t += Time.deltaTime;
        transform.localScale =
            Vector3.Lerp(Vector3.zero, escalaFinal, t / duracionAparicion);
        yield return null; // espera al siguiente frame
    }

    transform.localScale = escalaFinal;
}
}

```

Estudia este ejemplo

RutinaAparicion() es una coroutine resuelta que te servirá de modelo: guarda la escala final, parte de cero y va interpolando con Vector3.Lerp un poco cada frame usando yield return null. Las coroutines de la Parte B siguen patrones muy parecidos.

Paso 5 — Script GeneradorFiguras.cs (base PROVISTA)

Crea un GameObject vacío llamado Generador y agréale este script. Arrastra los prefabs de obstáculos al arreglo prefabsObstaculo y el prefab verde a prefabPoder. La coroutine RutinaOleadas() la completarás en la Parte B.

```

using System.Collections;
using UnityEngine;

public class GeneradorFiguras : MonoBehaviour
{
    [Header("Prefabs de figuras")]
    [SerializeField] private GameObject[] prefabsObstaculo; // circulo, triangulo
    [SerializeField] private GameObject prefabPoder;        // cuadrado verde (escudo)

    [Header("Zona de aparicion")]
    [SerializeField] private float xMin = -7f;
    [SerializeField] private float xMax = 7f;
    [SerializeField] private float alturaAparicion = 6f;
}

```

```

[Header("Dificultad")]
[SerializeField] private float intervaloInicial    = 1.2f;
[SerializeField] private float intervaloMinimo     = 0.35f;
[SerializeField] private float reduccionPorFigura  = 0.03f;
[SerializeField] private float probabilidadPoder   = 0.1f;

private bool generando = false;

// Metodo auxiliar PROVISTO: crea una figura en una posicion aleatoria.
private void GenerarFigura()
{
    float x = Random.Range(xMin, xMax);
    Vector3 posicion = new Vector3(x, alturaAparicion, 0f);

    GameObject prefab;
    if (Random.value < probabilidadPoder)
        prefab = prefabPoder;
    else
        prefab = prefabsObstaculo[Random.Range(0, prefabsObstaculo.Length)];

    Instantiate(prefab, posicion, Quaternion.identity);
}

public void Iniciar()
{
    if (!generando)
    {
        generando = true;
        StartCoroutine(RutinaOleadas());
    }
}

public void Detener()
{
    generando = false;
    StopAllCoroutines();
}

// ===== TAREA 1 (a completar por el estudiante) =====
private IEnumerator RutinaOleadas()
{
    // TODO: implementar segun la consigna
    yield break;
}
}

```

Paso 6 — Script Jugador.cs (base PROVISTA)

Agrega este script al objeto Jugador. El movimiento horizontal y la detección de colisiones ya están resueltos. Completaras RutinaDestello() y RutinaEscudo() en la Parte B.

```

using System.Collections;
using UnityEngine;

public class Jugador : MonoBehaviour
{
    [Header("Movimiento")]
    [SerializeField] private float velocidad = 8f;
    [SerializeField] private float limiteX   = 7f;

    [Header("Dano y escudo")]
    [SerializeField] private int   parpadeos      = 4;

```

```

[SerializeField] private float intervaloParpadeo = 0.1f;
[SerializeField] private float duracionEscudo    = 4f;
[SerializeField] private Color  colorEscudo      = Color.cyan;

private SpriteRenderer sr;
private Color colorNormal;
private bool  escudoActivo = false;

private void Awake()
{
    sr = GetComponent<SpriteRenderer>();
    colorNormal = sr.color;
}

private void Update()
{
    float h = Input.GetAxisRaw("Horizontal");
    Vector3 pos = transform.position;
    pos.x += h * velocidad * Time.deltaTime;
    pos.x = Mathf.Clamp(pos.x, -limiteX, limiteX);
    transform.position = pos;
}

private void OnTriggerEnter2D(Collider2D otro)
{
    if (otro.CompareTag("Poder"))
    {
        Destroy(otro.gameObject);
        StartCoroutine(RutinaEscudo(duracionEscudo));
    }
    else if (otro.CompareTag("Obstaculo"))
    {
        Destroy(otro.gameObject);
        if (escudoActivo) return;          // el escudo protege

        GestorJuego.instancia.PerderVida();
        StartCoroutine(RutinaDestello());
    }
}

// ===== TAREA 2 (a completar) =====
private IEnumerator RutinaDestello()
{
    // TODO
    yield break;
}

// ===== TAREA 3 (a completar) =====
private IEnumerator RutinaEscudo(float duracion)
{
    // TODO
    yield break;
}
}

```

Paso 7 — Interfaz y GestorJuego.cs (base PROVISTA)

1. Crea un Canvas (GameObject > UI > Canvas) y dentro dos textos TextMeshPro: uno grande al centro (Mensaje) para la cuenta regresiva y el Game Over, y otro en una esquina (Vidas).
2. Crea un GameObject vacío llamado GestorJuego, agrégale el script de abajo y arrastra a sus campos el Generador y los dos textos.

```
using System.Collections;
```

```
using UnityEngine;
using TMPro;

public class GestorJuego : MonoBehaviour
{
    public static GestorJuego instancia;

    [Header("Referencias")]
    [SerializeField] private GeneradorFiguras generador;
    [SerializeField] private TextMeshProUGUI textoMensaje;
    [SerializeField] private TextMeshProUGUI textoVidas;

    [Header("Configuracion")]
    [SerializeField] private int vidasIniciales = 3;

    private int vidas;
    private bool juegoActivo = false;

    private void Awake()
    {
        instancia = this;
    }

    private void Start()
    {
        vidas = vidasIniciales;
        ActualizarVidas();
        StartCoroutine(RutinaCuentaRegresiva());
    }

    public void PerderVida()
    {
        vidas--;
        ActualizarVidas();
        if (vidas <= 0)
            GameOver();
    }

    private void ActualizarVidas()
    {
        textoVidas.text = "Vidas: " + vidas;
    }

    private void GameOver()
    {
        juegoActivo = false;
        generador.Detener();
        textoMensaje.text = "GAME OVER";
    }

    // ===== TAREA 4 (a completar) =====
    private IEnumerator RutinaCuentaRegresiva()
    {
        // TODO
        yield break;
    }
}
```

Antes de pasar a la Parte B

Si presionas Play ahora, el juego no hará nada visible: las cuatro coroutines están vacías (yield break). Eso es lo que vas a resolver a continuación. El proyecto debe COMPILAR sin errores; si hay errores rojos en la consola, corrígelos antes de seguir.

IV. Parte B — Completa las coroutines (80 puntos)

Implementa los cuatro metodos marcados como TODO. Solo debes escribir el cuerpo de cada coroutine; no modifiques las firmas ni el resto del codigo provisto.

Tarea 1 — RutinaOleadas (Generador) · 25 pts

Archivo: GeneradorFiguras.cs. Esta coroutine debe generar figuras de forma indefinida mientras generando sea true, con dificultad creciente:

- Llama a GenerarFigura() para crear una figura.
- Espera intervalo segundos antes de generar la siguiente.
- El intervalo empieza en intervaloInicial y se reduce reduccionPorFigura cada vez, pero nunca debe ser menor que intervaloMinimo (usa Mathf.Max).

Tarea 2 — RutinaDestello (Jugador) · 25 pts

Archivo: Jugador.cs. Cuando el jugador recibe daño debe parpadear en rojo. La coroutine debe:

- Repetir parpadeos veces: poner el color en rojo, esperar intervaloParpadeo, volver al colorNormal y esperar intervaloParpadeo.
- Al terminar, el jugador debe quedar con su color normal.

Tarea 3 — RutinaEscudo (Jugador) · 25 pts

Archivo: Jugador.cs. Al recoger el poder verde el jugador queda protegido durante duración segundos. La coroutine debe:

- Marcar escudoActivo = true y cambiar el color del sprite a colorEscudo.
- Esperar duracion segundos.
- Restaurar colorNormal y poner escudoActivo = false.

Pista

En OnTriggerEnter2D (ya provisto) se ignora el dano cuando escudoActivo es true. Por eso basta con manejar bien la bandera y el color dentro de la coroutine.

Tarea 4 — RutinaCuentaRegresiva (GestorJuego) · 25 pts

Archivo: GestorJuego.cs. Antes de iniciar la partida debe mostrarse una cuenta regresiva. La coroutine debe:

- Mostrar en textoMensaje la secuencia "3", "2", "1", "¡YA!", esperando 1 segundo entre cada uno.
- Luego limpiar el mensaje (cadena vacia), poner juegoActivo = true y arrancar el generador con generador.Iniciar().

V. Resumen de puntaje

Seccion / Tarea	Puntos
IV. Tarea 1 — RutinaOleadas (dificultad creciente)	25
IV. Tarea 2 — RutinaDestello	25
IV. Tarea 3 — RutinaEscudo	25
IV. Tarea 4 — RutinaCuentaRegresiva	25
TOTAL	100

Checklist de funcionamiento (autoevaluación)

- ✓ Al presionar Play aparece la cuenta 3-2-1-¡YA! y recién entonces empiezan a caer figuras.
- ✓ Las figuras caen cada vez más seguido conforme avanza el tiempo.
- ✓ Al chocar con un obstáculo el jugador parpadea en rojo y pierde una vida.
- ✓ Al tomar el cuadrado verde el jugador cambia de color y no recibe daño por unos segundos.
- ✓ Al llegar a 0 vidas aparece GAME OVER y dejan de aparecer figuras.